Caesar Cipher Encryptor

AlgoExpert · Easy · Strings

The problem

- Shift each lowercase letter by `key` positions, wrapping around z.

Intuition

- Shift each letter by the key, wrapping around the alphabet with modulo twenty-six.

Brute force vs optimal

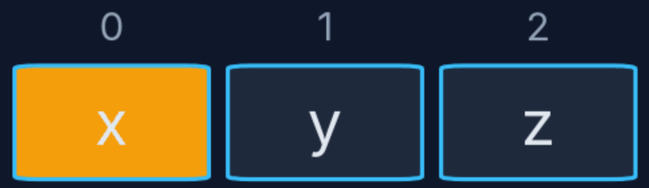
- Naive: shift one letter step by step k times → O(n·k)
- Optimal: arithmetic with modulo 26 → O(n)
- Reduce k mod 26 first (k can be huge)

Approach

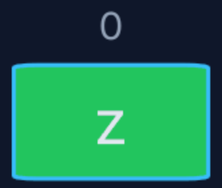
- 1. Reduce the key modulo twenty-six
- 2. For each letter, compute its new code
- 3. Wrap past z back to a
- 4. Build the new string

Worked example

input



output

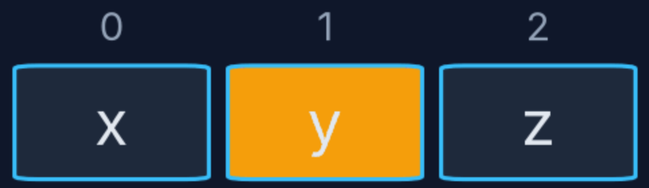



```
'x' = 23; (23 + 2) mod 26 = 25 → 'z'
```

'x' shifted by 2 → 'z'

Worked example

input



output

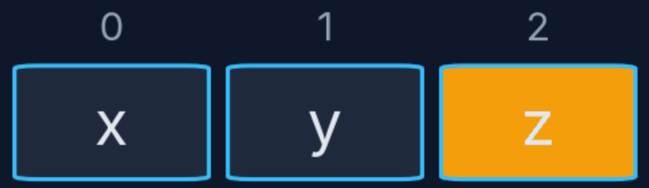



```
'y' = 24; (24 + 2) mod 26 = 0 → 'a'
```

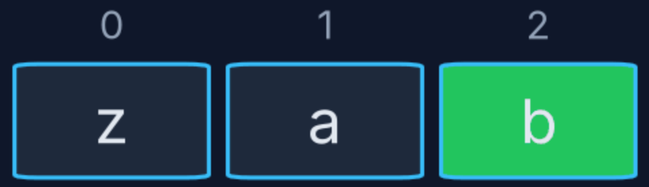
'y' shifted by 2 → 'a'

Worked example

input



output




```
'z' = 25; (25 + 2) mod 26 = 1 → 'b'
```

'z' shifted by 2 → 'b'

Java solution

```java
public class CaesarCipherEncryptor {
    public static String caesarCypherEncryptor(String str, int key) {
        int k = key % 26;
        StringBuilder sb = new StringBuilder();
        for (char c : str.toCharArray()) sb.append((char) ('a' + (c - 'a' + k) % 26));
        return sb.toString();
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- Wrap-around: 'z' + 1 = 'a'
- Large k (e.g. 54 ≡ 2)
- Lowercase only per problem

Interview tips & follow-ups

- Use a StringBuilder / char array, not string concatenation
- Follow-up: decrypt, uppercase, non-letters

Recap

- Shift each letter by the key, wrapping around the alphabet with modulo twenty-six.
- Complexity: O(n)